



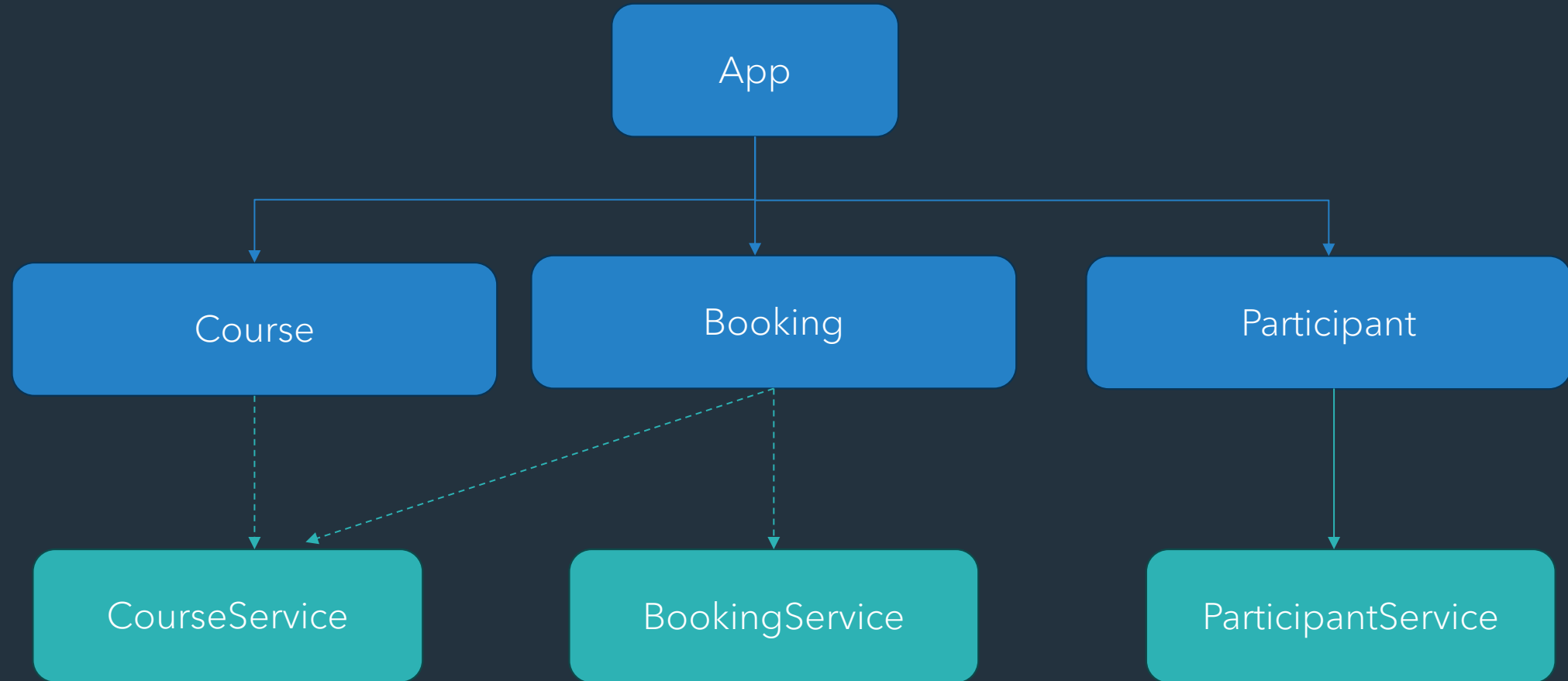
STATE
MANAGEMENT

Ngrx

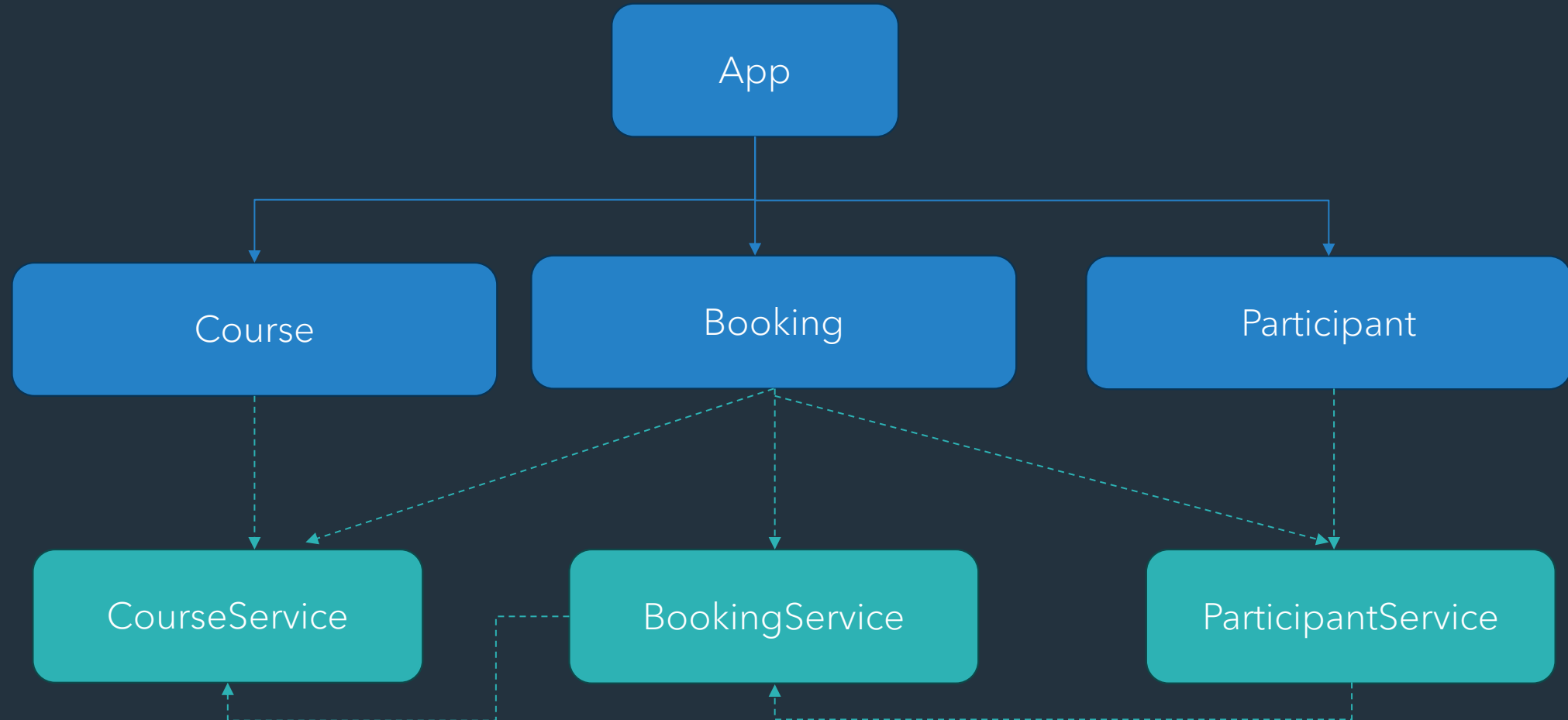
INHALT

- @ngrx/store als Redux Implementierung
- Actions
- Reducer
- Selektoren
- Effects

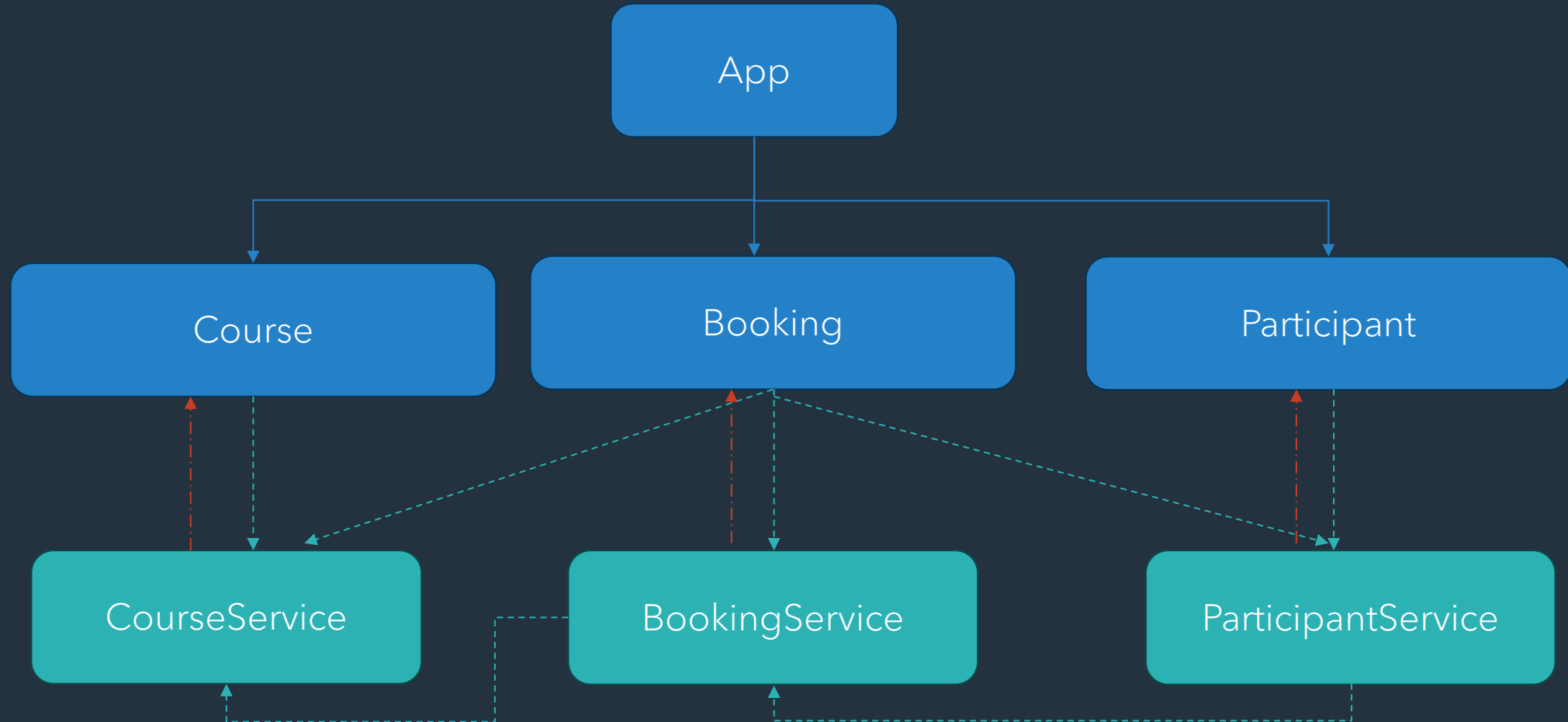
ZUSTANDSVERWALTUNG – V1



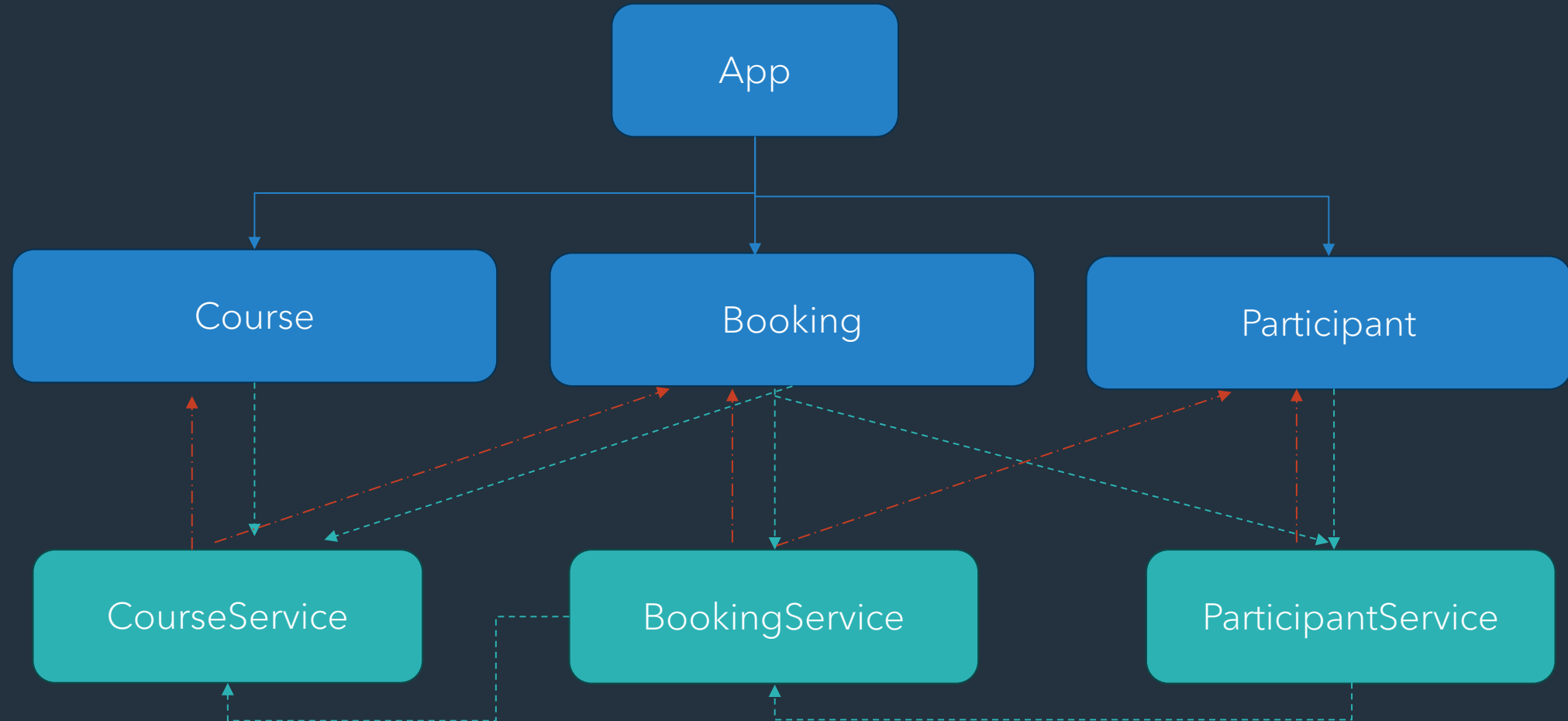
ZUSTANDSVERWALTUNG – V2



ZUSTANDSVERWALTUNG – V3

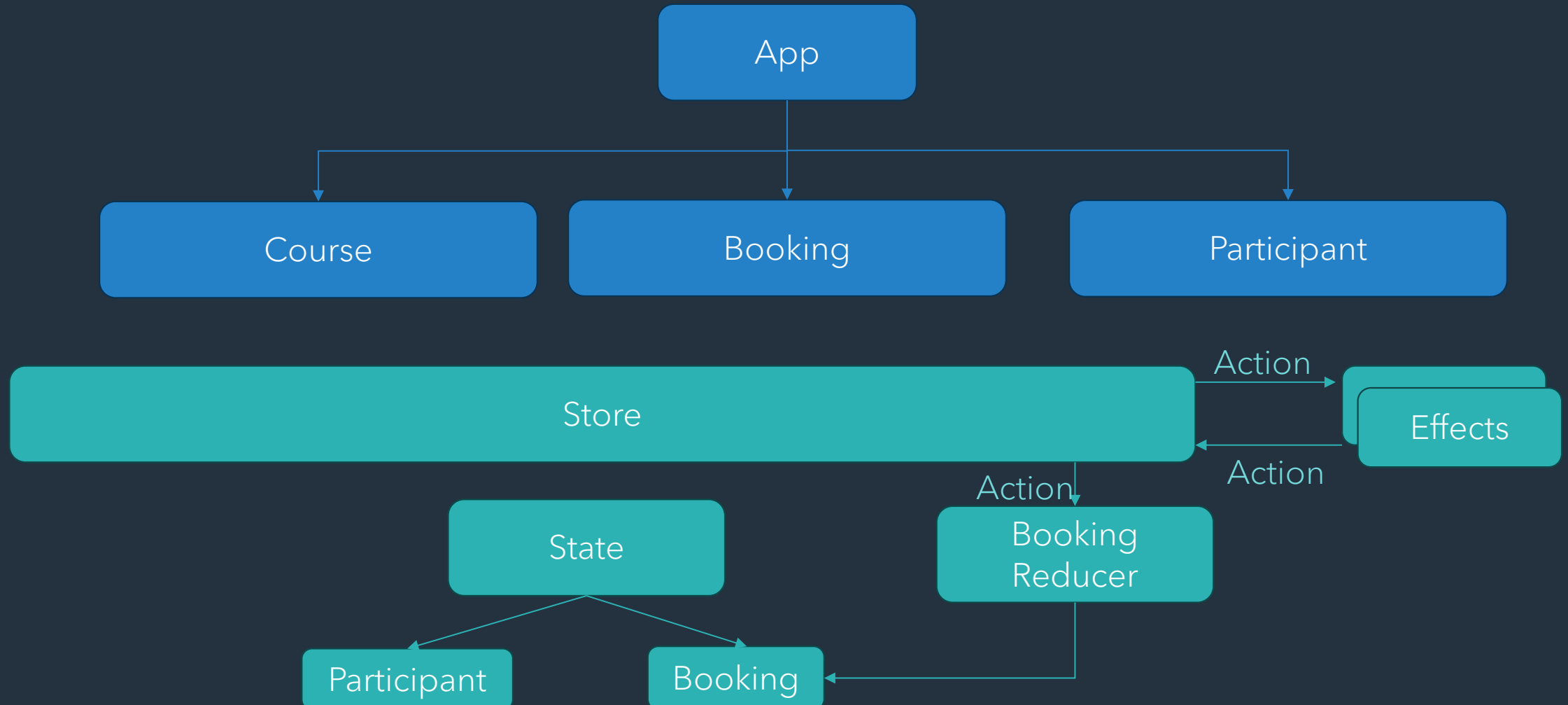


ZUSTANDSVERWALTUNG – V3.1





ZUSTANDSVERWALTUNG – V3.1



STORE

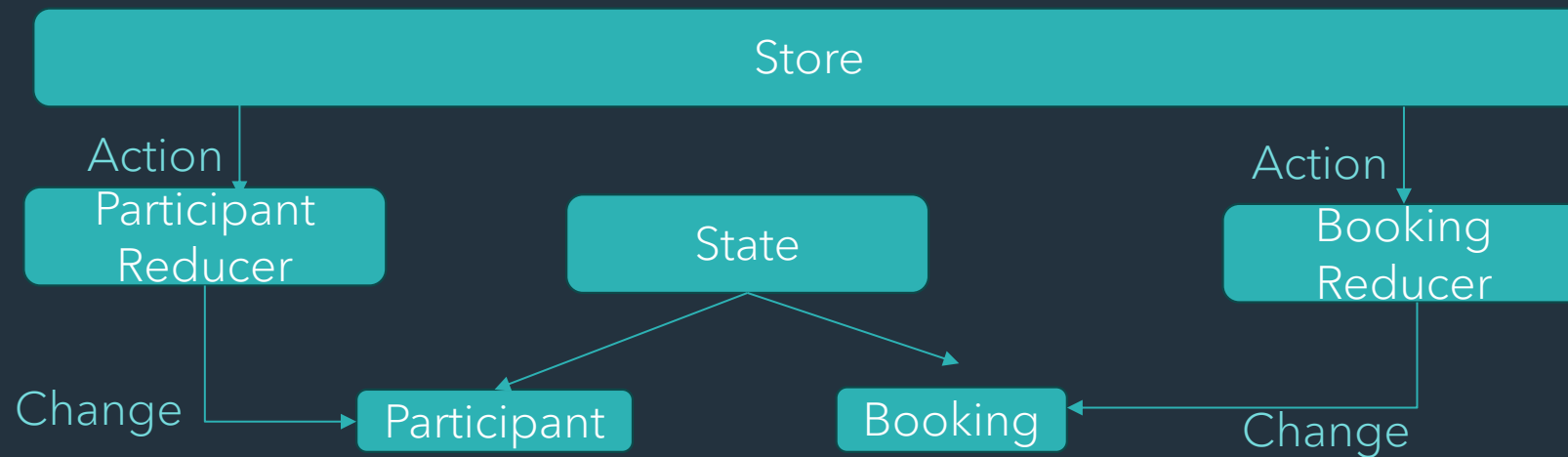
- verwaltet den Anwendungszustand
- *Objektbaum Struktur*
- *Unveränderbarer Zustandsbaum (NGRX) immutable*
- *Änderung durch Kopien*
- *Change Management Vergleich der Objektreferenzen (Performance).*
- *Observables für Änderungs-Notification-Events*

ACTIONS - DML

- Datenmanipulation mittels Actions
- Konsistenz wird gewahrt
- Vergleichbar mit Aufruf einer DB-Procedure
- Verarbeitung der Actions mittels Reducers

REDUCER

- Funktionen zur **Bearbeitung** von Actions
- Verantwortlich für ein **Ast** des Zustandbaums
- Jeder Reducer erhält jede Action (Zyklenvermeidung)
- Laufen synchron



EFFECTS - ASYNCHRONE TASKS

- Erfordert eine Action einen Asynchronen Teil
- Kommen Effects zu Einsatz
- Return-Action nach Bearbeitung eines Effects

Erfolgsfall oder

Misserfolgsfall

NGRX EINRICHTEN

- `ng add @ngrx/store`
- `main.ts` (Standalone):
`bootstrapApplication(AppComponent, { providers: [provideStore(),
provideStoreDevtools()] })`
- Legacy AppModule: `imports: [ StoreModule.forRoot({}, {}) ]`

NGRX EINRICHTEN – EFFECTS

- `ng add @ngrx/effects`
- `main.ts` (Standalone):
`providers: [ provideStore(), provideEffects() ]`
- Legacy AppModule: `imports: [ EffectsModule.forRoot([]) ]`

FEATURE ERZEUGEN

- `ng g @ngrx/schematics:feature pig/pig --api false --flat false`
- Argumente
- Name des Features (Entity)
- Ohne `--module`: Registrierung per `provideState()` (nächste Folie)
- Keine BackendCalls
- Flat false damit die generierten Dateien im pig Package landen

FEATURE REGISTRIEREN

providers: [provideState(fromPig.pigFeatureKey, fromPig.reducer), provideEffects(PigEffects)] // an der Route 'pig' oder in main.ts

export const pigFeatureKey = 'pig'; // AUS PigReducer

export const reducer = createReducer(
 initialState,
 on(PigActions.loadPigs, state => state),
);

Grundgerüst Reducer

loadPigs\$ = createEffect(() => {
 return this.actions\$.pipe(ofType(PigActions.loadPigs),
 // An EMPTY observable only emits completion. Replace with your own observable API request
 concatMap(() => EMPTY as Observable<{ type: string }>)
);
});

Grundgerüst Effect

STATE ANPASSEN → REDUCER KLASSE

```
export interface State {  
  pigs: Pig[];  
  loading: boolean;  
  error: unknown;  
}  
  
export const initialState: State = {  
  pigs: [],  
  loading: false,  
  error: {}  
};
```

pig.reducer.ts

- Neue Attribute für
- content,
- flag für loading
- und error

ACTIONS FESTLEGEN

```
on(PigActions.loadPigs, (state, action) => {  
  const pigs = [] as Pig[];  
  const loading = false;  
  return {...state, pigs, loading};  
}),  
  
on(PigActions.loadPigsSuccessful, (state, action) => {  
  const pigs = action.pigs;  
  const loading = false;  
  return {...state, pigs, loading};  
}),  
  
on(PigActions.loadPigsError, (state, action) => {  
  const pigs = [] as Pig[];  
  const loading = false;  
  const error = action.error;  
  return {...state, pigs, loading, error};  
}),
```

- Reducer Funktionen
(state, action) => state
- State ist immutable daher neuer Erzeugen
- loadPigs, warum wird nicht geladen?
Das Laden der Daten ist async → Effect

EFFECTS – ASNYC DATENBESCHAFFER

```
@Injectable()
export class PigEffects {
  private readonly actions$ = inject(Actions);
  private readonly pigService = inject(PigService);
  loadPigs$ = createEffect(() => {
    return this.actions$.pipe(
      ofType(PigActions.loadPigs),
      /** Call PigService; success -> loadPigsSuccessful, else loadPigsError */
      switchMap(() => this.pigService.getAll().pipe(
        map(pigs => PigActions.loadPigsSuccessful({ pigs })),
        catchError(error => of(PigActions.loadPigsError({ error })))
      ))
    );
  });
}
```



- Logik zum Laden der Daten für den Store
- Z.B. Service Call gegen Backend
- Hier nur loadPigs Action

ZUGRIFF AUF STORE

```
export class PigListComponent implements OnInit {  
  private readonly store = inject(Store);  
  pigs = this.store.selectSignal(selectPigs); // Signal statt  
Observable  
  ngOnInit(): void {  
    this.store.dispatch(PigActions.loadPigs());  
  }  
}
```

- Select
select() liefert ein Observable,
selectSignal() ein Signal
- dispatch
*schickt das Event, um die Daten initial
zu holen*

AUSBLICK: NGRX SIGNAL STORE

- `npm install @ngrx/signals`

```
export const PigStore = signalStore(  
  { providedIn: 'root' },  
  withState({ pigs: [] as Pig[], loading: false }),  
  withComputed(({ pigs }) => ({ count: computed(() => pigs().length)  
})),  
  withMethods((store, service = inject(PigService)) => ({  
    load() { ... patchState(store, { pigs }) ... }  
  })))  
);
```

- Verwendung: `readonly store = inject(PigStore);` im Template `store.pigs()`